

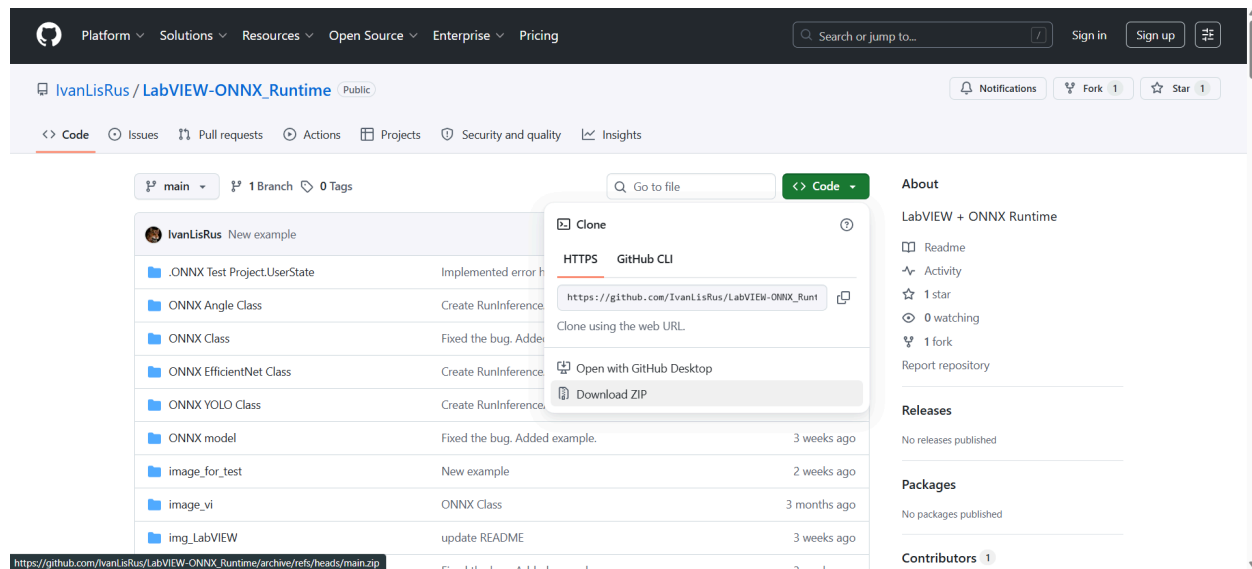
Guide for Ivan's method configuration

“Ivan's method” is a procedure in which we use LabVIEW libraries from a public GitHub created by a user named Ivan. This libraries let us open our ONNX model file and predict with it directly inside LabVIEW, thing that presents an advantage over the Python Node, making the program faster since there is no necessity of starting up the python session or passing information from LabVIEW to Python, obtaining significantly faster times than the Python Node method. In this guide, we will review all the significant steps in obtaining a working LabVIEW program that reads an ONNX file, obtains data for prediction from LabVIEW, and delivers an output voltage.

1. Downloading Ivan's GitHub repository

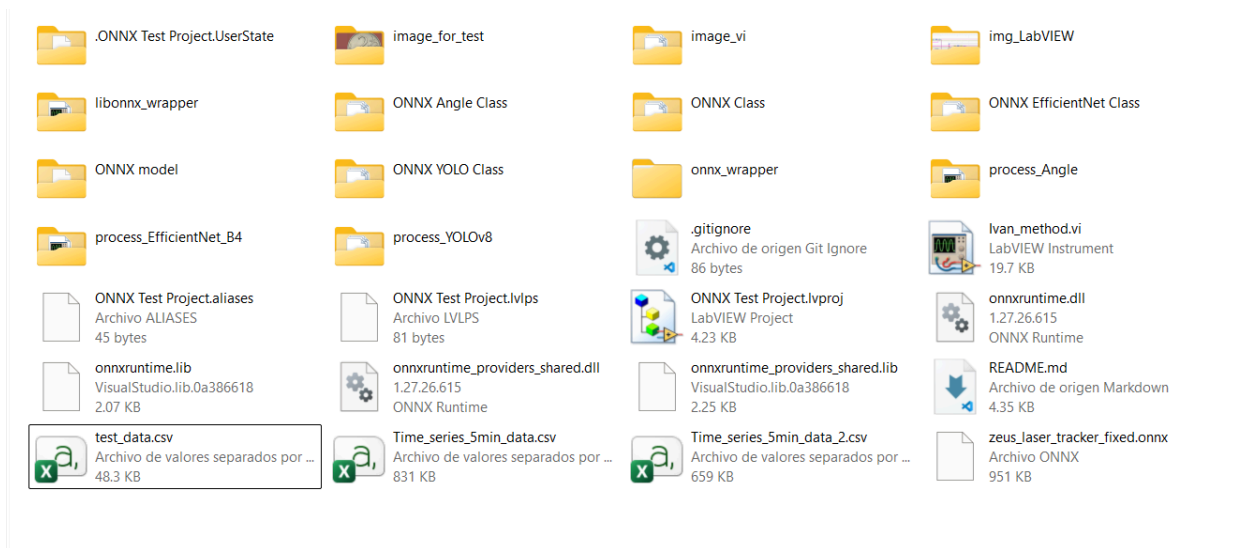
Link: https://github.com/IvanLisRus/LabVIEW-ONNX_Runtime

Obviously, the first step is to download the .zip from the repository LabVIEW-ONNX_Runtime. Remember that to do this you enter the link above, then click on the green “code” button and finally select “Download ZIP” as shown in the image below.



This will give you a local ZIP that has the exact structure of the repository. You need to extract the zip in order to have everything you need. If you look into the folder, you will see you have several subfolders that contain the different classes this code will use. It also has examples and images that help to understand the whole purpose or functioning of this code, but it is not necessary to look into it, as the whole scope of Ivan's project is outside of what we want to do. You can also see that we have several .vi and a project that contains these VIs. It is recommended that you erase all VIs and leave the project with only a new blank VI created by you, although it is not necessary.

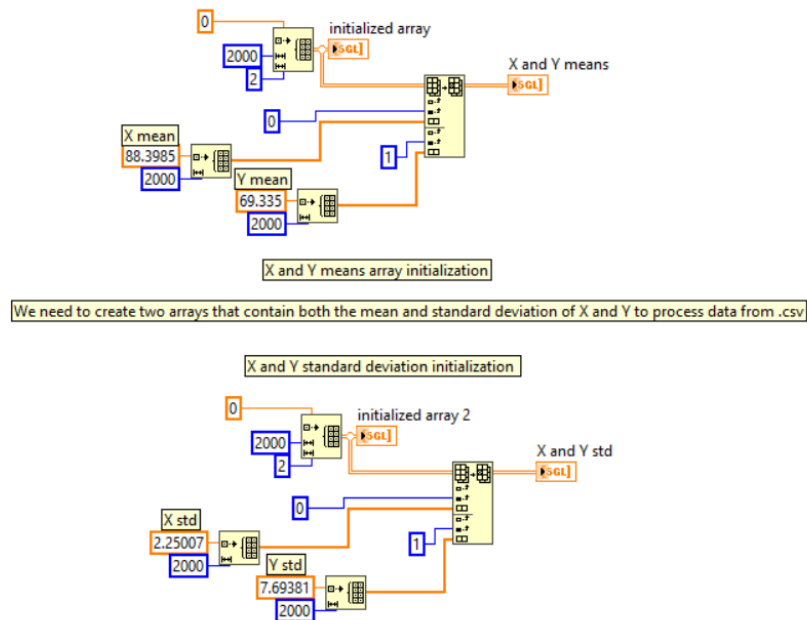
Finally, to fully have your folder setup, you need to add to this folder your .onnx file, and move the wrapper files from the folder “onnx_wrapper” outside just to be sure these files are read by the LabVIEW code. For this experiment, the final folder looked like the image below, having all sub-folders intact, erasing all VIs, and adding the .onnx file, the “onnx_wrapper” files, as well as the .csv needed as input.



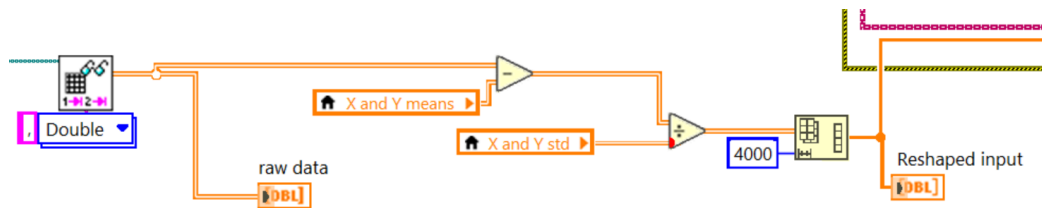
It is important to know that for this method to work you need a 64-bit version of LabVIEW, since the wrappers for the libraries are all 64-bit

Reading and preprocessing data

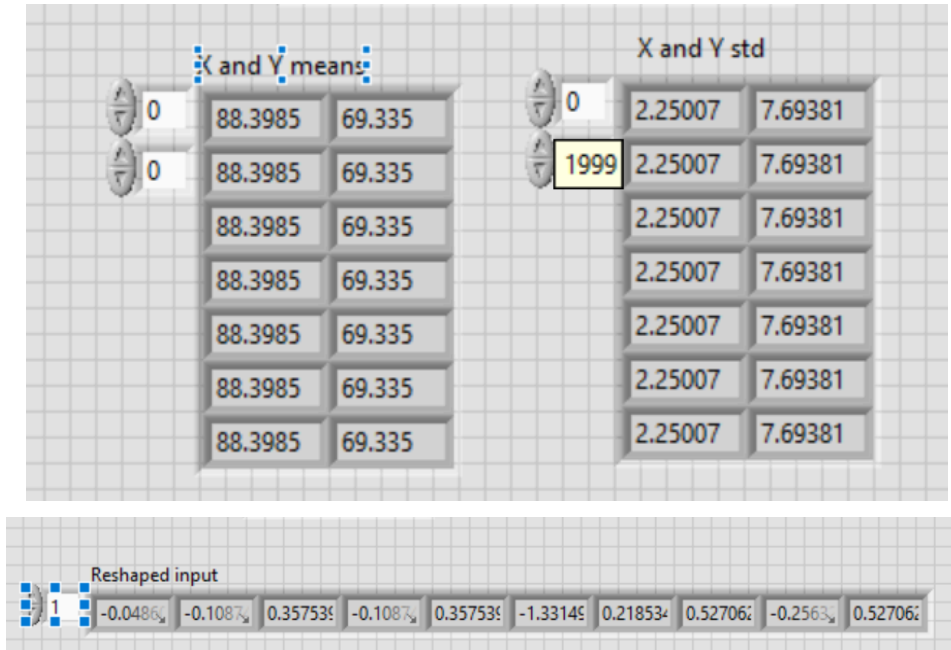
We start our LabVIEW program by reading the data from our .csv as always with a Read Delimited Spreadsheet, just wiring the file path. Remember this part will change as we end up reading the data from real time sensing. For this case, it is necessary to pre-process our data because our model is expecting normalized data. We will need to subtract the mean of X and Y positions and then divide the result by the standard deviation in order to obtain the information our model needs (as always, this will probably change depending on the data read and model used). To be able to do this process to every single point, two 2D arrays were created, in which we store the means and std of both X and Y. The image below shows the blocks and configuration used for this experiment. Since we have a 2000 points input, we need 2000 points of X and Y means and std, therefore the size of the array is established.



The next image shows the procedure of subtracting and dividing our X and Y positions by the mean and std. As you can see, the final step of preprocessing is to reshape our 2D array into a 1D array of 4000 points, as this is the shape that the model expects.

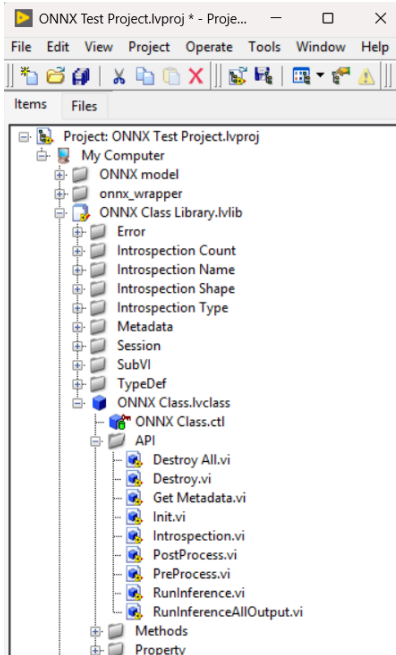


Finally, you can confirm your preprocessing works by running the code and checking added indicators for each step and array of the process.

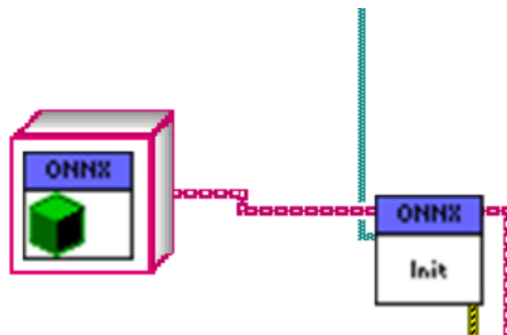


Opening ONNX model and Predicting

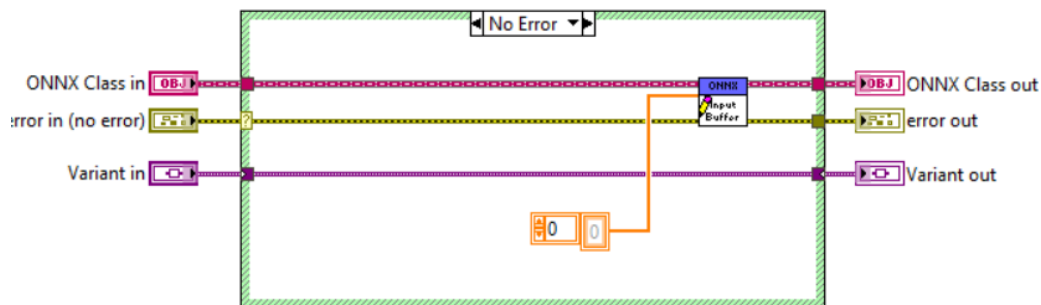
After preprocessing, we are ready to give the input to our model and predict the X and Y coordinates. To do this, we need 3 VIs from a specific library from Ivan's repository. The three programs we will use are Init.vi, PreProcess.vi, and RunInference.vi. These files can be found inside the project, with the next route, also shown in the image below: ONNX Class Library - ONNX Class - API.



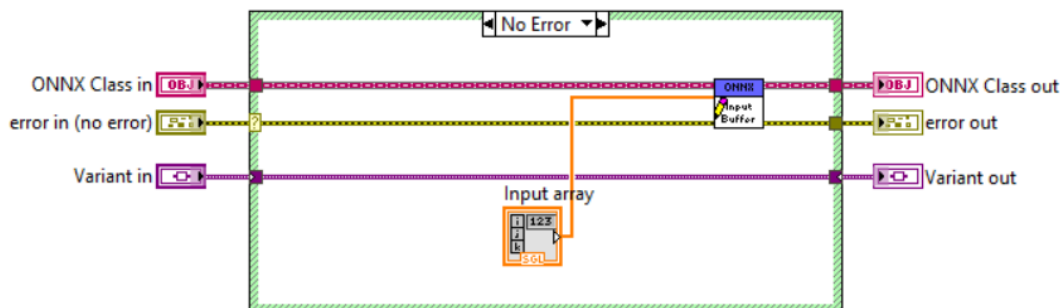
To configure the Init.vi, we need to wire the terminal ONNX path and write the path in which our model is in. It is also necessary to create and wire a constant to the terminal ONNX Class In. We will also wire the ONNX Class Out and Error Out from Init.vi to the ONNX Class and Error In from the PreProcess.vi block. All wired, the block should look like the image below.



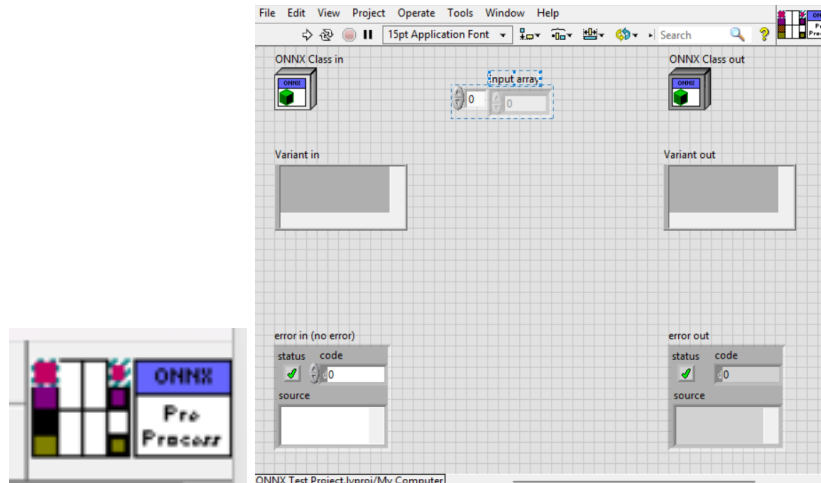
The configuration of the PreProcess.vi block is actually complex, as we have to modify the original version. To modify, double click the VIs block and open the program. Go to the backend, and it should look like the image below. As it stands, PreProcess.vi is feeding an empty array into our ONNX model, not our real data.



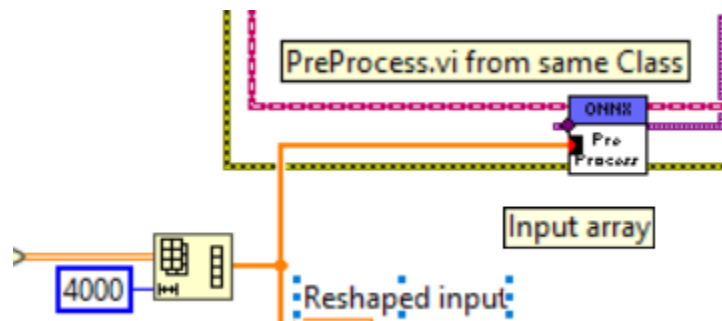
To fix this, start by deleting the array constant, and create an array control in your front end. Then you will wire your new array control into the InputBuffer sub-VI, like in the image below.



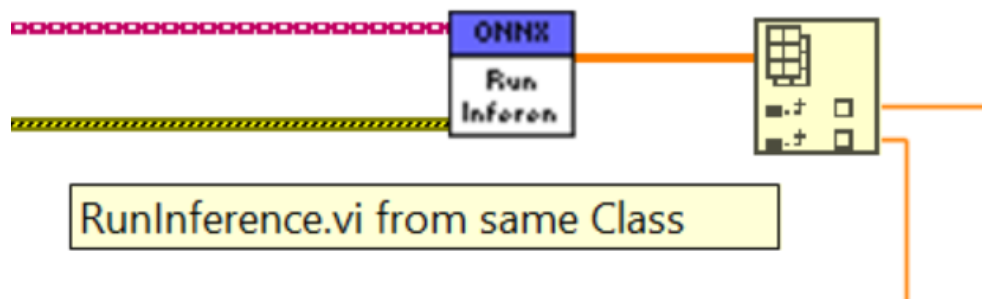
Finally we need to add a terminal into the PreProcess block for the new array control. To do this, go to the front end and look at the top right corner of your page. Here you will see both the icon for the VI block and the connector pane, where you can add a terminal. Select an empty block in the connector pane, and this is where your new terminal will be added. Finally, you need to click on your array control, and this will create a new terminal in your PreProcess.vi block that is connected to the array control.



Now, go back into your main VI, and wire your reshaped input into this new Input array terminal. **NOTE: The main program will mark an error because dependencies changed on the YOLO, EffcientNet, and Angle class, delete this classes, save and close the whole project, and open it again, it should mark everything correct now.**

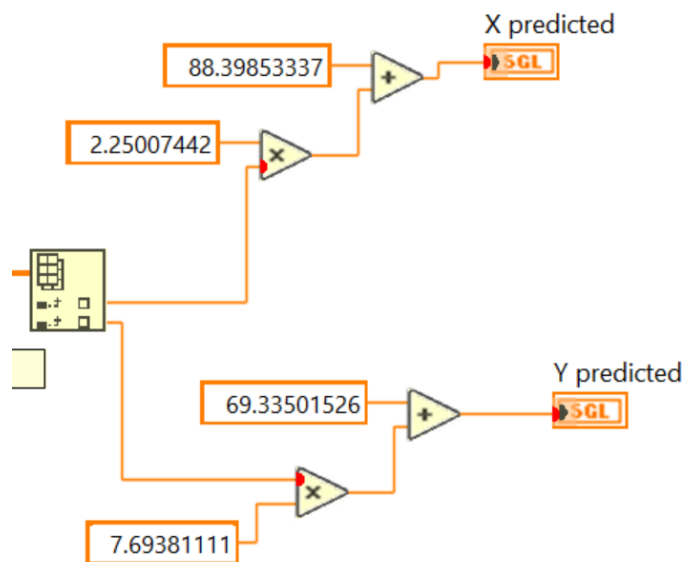


For the RunInference.vi block we don't need to wire anything in aside from the mentioned before, but we need to wire our Output Buffer terminal into a Index Array block in order to read the predicted X and y coordinates.

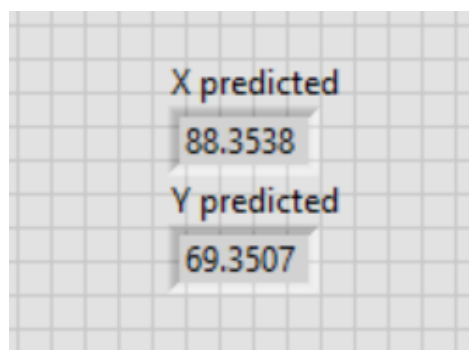


PostProcessing

Because our predictions are made based on normalized data, it is necessary to make a postprocessing to pass out normalized predictions to real X and Y coordinates. To do this we just need to wire our X and Y coordinates from the Index Array into Multiply and then Sum nodes, in which we multiply by the std and then sum the mean. We can add an indicator to the result of both sums and obtain our predicted X and Y coordinates. This last part should look like the image below



With all this you should be able to run your program and obtain predicted X and Y coordinates, like shown in the indicators below.



The screenshot displays a LabVIEW block diagram for a neural network model. The diagram is organized into several functional sections:

- Path for the .onnx Model:** Located at the top, it points to the file path: `C:\Users\PSCUser\Documents\LabVIEW Data\Testing info\LabVIEW-ONNX_Runtime-main\LabVIEW-ONNX_Runtime-main\zeus_laser_tracker_fixed.onnx`.
- Path for test data (only in experiment):** Located on the left, it points to the file path: `C:\Users\PSCUser\Documents\LabVIEW Data\Testing info\LabVIEW-ONNX_Runtime-main\LabVIEW-ONNX_Runtime-main\test_data.csv`.
- Initialization and Data Loading:**
 - A `Double` control is set to `1.2`.
 - A `raw data` array is loaded from the test data path.
 - An `initialized array` is created with a size of `2000`.
 - The `X and Y means` are calculated and stored in an array.
- Preprocessing:**
 - The `PreProcess.vi` from the same class is used to process the raw data.
 - The `PreProcess` block has a `Variant out` indicator.
- Inference:**
 - The `RunInference.vi` from the same class is used to perform the inference.
 - The `RunInference` block has a `Reshaped input` indicator.
- Output Calculation:**
 - The `X predicted` value is calculated as `88.39853337`.
 - The `Y predicted` value is calculated as `7.69381111`.

Annotations and comments provide additional context:

- "We subtract mean of X and Y, then divide by the std"
- "We need to reshape our 2D array into a 1D array with the 4000 datapoints"
- "We multiply the prediction by the std and sum the mean to get the real prediction value for both X and Y"
- "X predicted"
- "Y predicted"

